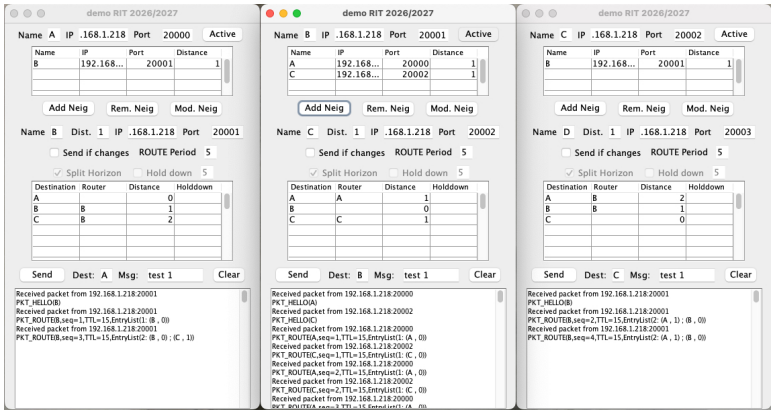
NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING



Integrated Telecommunication Networks (RIT)

2026/2027

1st Laboratory work:
Dynamic Routing in a Mesh Network

Classes 2 to 5

Mestrado em Engenharia Eletrotécnica e de Computadores / MERSIM

https://tele1.deec.fct.unl.pt/rit

Luis Bernardo

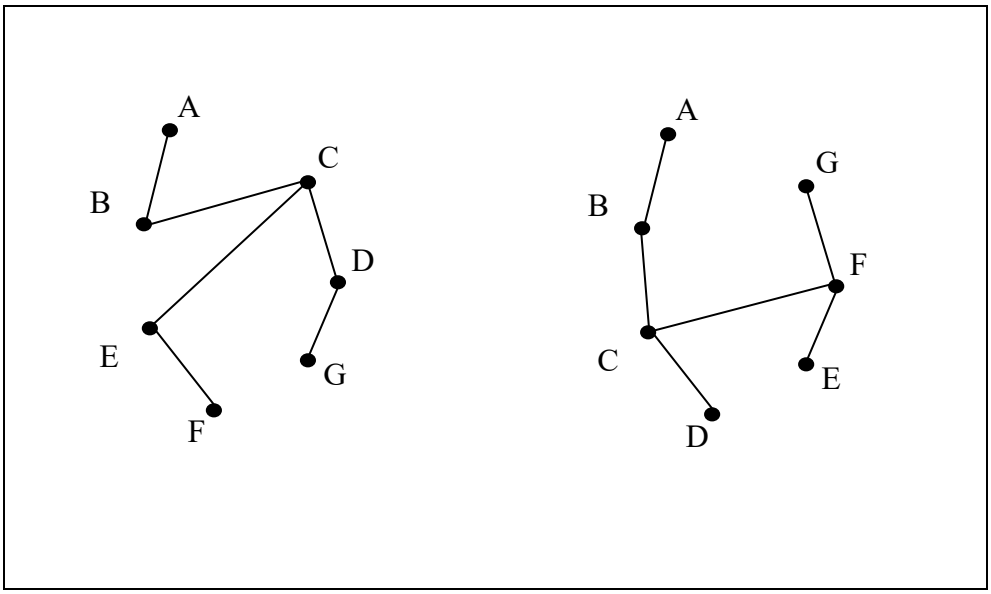
1. Objectives

Familiarisation with the operation of routers in a mesh network. The work involves communication between interconnected components in a mesh network. It is assumed that the stations utilise a mesh network on top of the physical Ethernet network. The work consists of the development of a ***router program*** that can communicate with neighbouring *routers*, providing a routing service based on the **distance-vector routing protocol**. The algorithm is derived from a simplification of the IGRP (*Inter-Gateway Routing Protocol*), proposed by Cisco, used in TCP/IP networks.

In real systems, routers interconnect machines belonging to several local networks. In this work, we will simulate this environment with some simplifications: the routers are simultaneously the senders and receivers of messages. By generating packets from various routers and sending them to all other routers, the work will enable testing the behaviour of the routing service in the face of changes in the network.

2. Specifications

Each *router process* on the network is identified by a name: A, B, C, D, ..., Z. Although you are using an Ethernet network in the Lab or a computer at home, from a logical point of view, routers cannot communicate directly with each other. They have a circuit to do so. The topology of each mesh network is defined by the set of neighbourhood relationships introduced locally in the graphical interface of each *router*. Each *router* is only aware of its direct neighbours, receiving information about the rest of the network only through the routing algorithm used – the distance vector. The figures below show two hypotheses of networks using 7 *routers*.



The various ***router*** processes of each machine communicate through *datagram* sockets.

An actual router monitors its neighbours on the network and periodically exchanges control packets (with routing tables) with them. In this simulation, the router processes simulate this behaviour by using commands in the user interface.

A router provides the user with options for monitoring neighbours:

- allows it to add neighbours (identified by the IP address and port of their socket);
- allows it to change the distance to any of the neighbours;
- allows it to close a connection to a neighbour.

The *router* process periodically sends packets with local routing information to neighbours and recalculates its local table using the distance-vector algorithm. To make the router's status easier to see, the contents of the local routing table are displayed in the graphical interface.

Finally, the *router process* offers options for sending and receiving data packets on its interface:

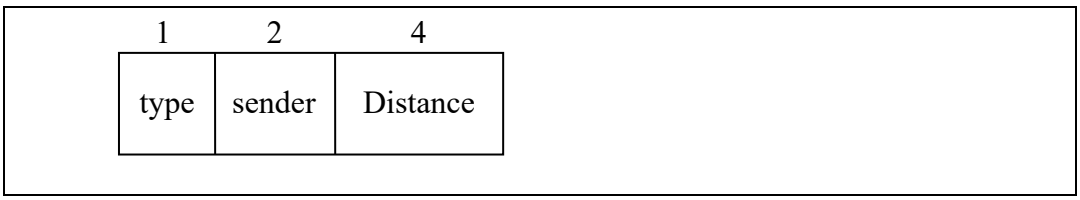
- allows sending data packets to any destination;
- receives and resends the data packets to a neighbouring router, depending on the contents of the routing table, and the path is stored in the packet;
- receives the data packets at the *destination router*, showing the complete path the packet took from source to destination.

Note that the network topology can differ for each direction in this simulator, as in a real system. For example, on a network access line with a heavily accessed web server, traffic will be higher in the direction originating from the web server.

2.1. Network Configuration

When the router program starts, it knows its address and has no neighbours. It will then gain or lose neighbours as the user uses add, remove, or modify distances, or as other routers associate with it. A protocol is used to create and destroy neighbour relationships between routers. When a user adds a neighbour to the list, the program sends a HELLO packet to the neighbour's IP address and port. When a router program receives a HELLO packet, it adds the packet sender to the neighbour list, memorises the list of neighbours' areas, and responds with a HELLO packet containing the same distance value and the areas to which the router belongs. By default, the distance is assumed to be the same in both directions, although the user can change the distance in each direction at any time. If the neighbour does not accept the HELLO packet because it exceeds the maximum number of neighbours, it terminates the neighbourhood relation by sending a BYE packet (see below).

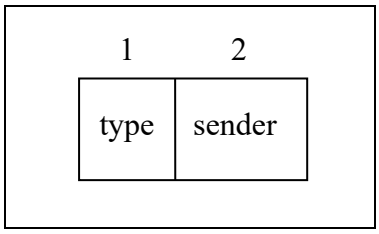
```
HELLO packet: { sequence of }
 byte       type;          { packet type  -  HELLO = 10 }
 char       sender;        { sender router name }
 int        distance;      { distance to the neighbour }
```



The HELLO packet is also used to communicate to a neighbour that the distance between the two routers has changed. The distance is an integer value between 0 and 20. **The distance 20 is reserved and symbolizes the non-existence of connection between two routers.** Distances greater than 20 or less than 0 should not be used.

When a router wants to terminate a neighbour relationship, it sends a BYE packet to the neighbour. Both the sender and the receiver of the message should immediately terminate sending packets with the link state (ROUTE) to the ex-neighbour.

```
BYE packet: { sequence of }
 byte       type;          { packet type  -  BYE = 20 }
 char       sender;        { sender router name }
```



During the work, it is intended to test the behaviour of the distance vector algorithms in response to the emergence of new routers and the termination of connections or routers. Thus, the behaviour of the developed program in these scenarios should be tested during the development of the work, as this behaviour will be evaluated in the final discussion. Notice that routers might fail silently (without sending BYE) and the system should detect this type of failure.

2.2. Routing protocol

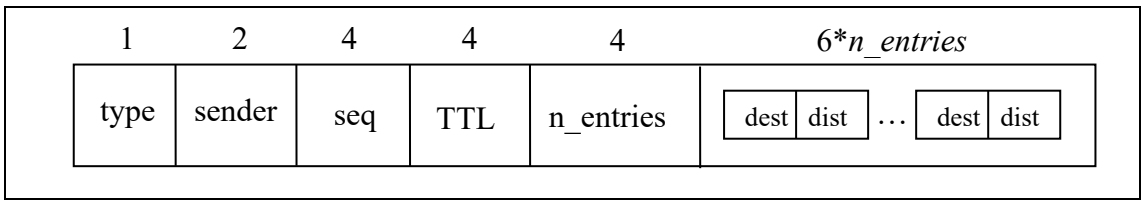
The router program's most important task is calculating the routing table using the distance-vector algorithm. From the moment it becomes active, the *router* program must periodically send a ROUTE packet to all neighbours, with the contents of its local routing table. Before sending a ROUTE packet, the *router* must recalculate its routing table using information from ROUTE packets received from neighbours.

Routers always store the most recent ROUTE packet from each neighboring routers, up to a maximum time equal to the packet lifetime (TTL). The TTL value is **equal to the duration of the ROUTE packet sending period plus ten seconds**.

ROUTE packet: { sequence of }

```
 byte        type;          { packet type  -  ROUTE = 30 }
 char        sender;        { sender router name }
 int         seq;           { sequence number }
 int         TTL;           { TTL [s] }
 int         n_entries;     { number of entries }
 Entry[]     entries;       { array with n_entries entries }

class Entry { // Type of array element
 char        dest;          { name of the neighbour router }
 int         dist;          { distance to the router - between 0 and 20}
}
```



To simplify the program, you must admit that the only modifications to the network topology (connections between neighbours) occur as a result of modifications in the user-controlled neighbourhood, or as a result of receiving HELLO and BYE packets. Assume no packet losses in the supporting local network where the applications run.

2.3. Handling the slow convergence problem

The distance vector routing algorithm converges slowly after the failure of a link or a router – which can give rise to the problem of counting to infinity. To deal with this problem, a number of partial solutions have been proposed to accelerate convergence.

One of the most widespread solutions is called "*Split Horizon*". If a router A uses a route to E that passes through B, then A announces to neighboring router B that it knows of no route to E **by omitting the destination in the sent vector**. This guarantees that B will never choose A to reach E, solving the convergence problem in a linear network without cycles. However, if the network has closed loops, the algorithm will still suffer from slow convergence.

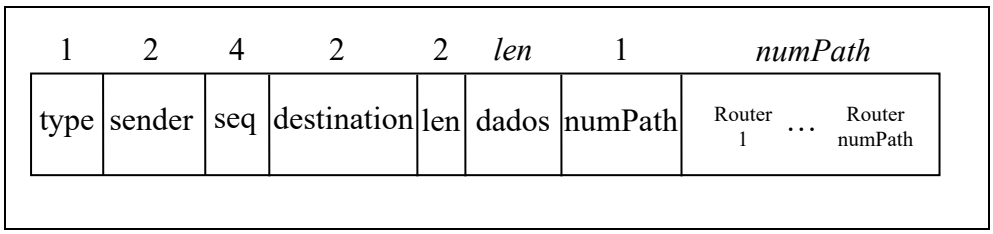
Another solution is called "*Hold down*". In this case, after a fault is detected in a route to a destination, the router will hold the infinity value in the table for a fixed time interval (spreading the infinity value to neighbours) before starting to search for a new route to that destination, allowing time for the infinite value to propagate across the network.

In this practical work, we will implement both algorithms and evaluate how effective they are at dealing with the counting-to-infinity problem.

2.4. Sending and receiving data

Router programs also simulate the role of the sender and receiver of data packets. Whenever the user selects the option to send data, the program must create a packet of type DATA, with the path value equal to the local name. The program must then query the routing table and send the packet to the next router indicated in the table or return the error code if the destination name is unreachable.

```
DATA packet: { contiguous sequence of }
 byte       type;         { packet type  -  DATA = 99 }
 Address    sender;       { origin router name }
 int        seq;          { sequence number }
 char       destination;  { destination router name }
 short      len;          { number of bytes of data }
 byte[]     dados[];      { len data bytes }
 byte       numPath;      { number of routers traversed }
 char[]     path;         { sequence of routers traversed }
```



Each *router* that receives the packet appends its name to the *packet*'s path field and increments *numPath*. If it is the packet destination router, it must write the contents of the received packet to the screen. Otherwise, the router should consult its routing table and send the packet to the next *router*. If the path length taken by the packet reaches the **maximum distance (10),** then an error message should appear, signalling this to the user, and the packet should be treated as if it had reached its destination. In this way, you always know where it has travelled.

3. Program Development

The work must be developed using the Apache Netbeans IDE development environment and with JDK 21. Other IDEs can be used, but the possibility of editing the GUI through the Netbeans editor is lost.

On the other hand, the **`git`** tool must be used during project development to save intermediate versions, making it possible to recover code deleted by mistake afterward.

It is suggested that students create a github.com account[1] and a private project to store the work done, in order to safeguard against failures in the machine used in the development of the project.

3.1. Application installation

The *router* application should be installed from the *github.com* cloning the project that is in:
https://github.com/lflbernardofct/router_rit.git .
Apache Netbeans supports this functionality by selecting the options *"Team"* > *"Git"* > *"Clone..."* indicating the URL of the repository and selecting the directory of the local computer where the code will be placed[2].

3.2. Software version management

Version management is a very important subject when developing software. The evolution of versions throughout development can be linear, or it can be more complex allowing you to have divergent paths and come together at some point in the future. Several powerful version management systems exist. One of them is **`git`**, which is integrated with *github.com*. Netbeans allows you to natively use **`git`** to save multiple intermediate versions of your work.

When you install the project from *github.com*, **`git`** configuration happens right away. If you cannot install the project this way, you need to initialize **`git`** for the project you installed from source. The tutorial below describes the configuration, but it is very simple: right-click the project (in this case, the router project) and select "Versioning" > "Initialize Git Repository...". Next, save the first version of the work (using commit). This defines the existing project as the project's base version.

During development, you must commit (by selecting the "*Git*" option > "*Commit ...*" in the project) to save relevant intermediate versions of the project. The lab assignment indicates some commits must be made and which names must be associated with them. In more advanced use, you can also create new project branches ("*branches*"), which you can later merge into the initial project.

Using the project's "**`git`**" option (with the right mouse button), you can see what has changed between versions or revert changes. It is very useful when something stops working.

It is recommended that you check out the following tutorial to learn how to use version control features of **`git`** in Netbeans (all the mechanisms are very intuitive):
https://netbeans.apache.org/tutorial/main/kb/docs/ide/git/

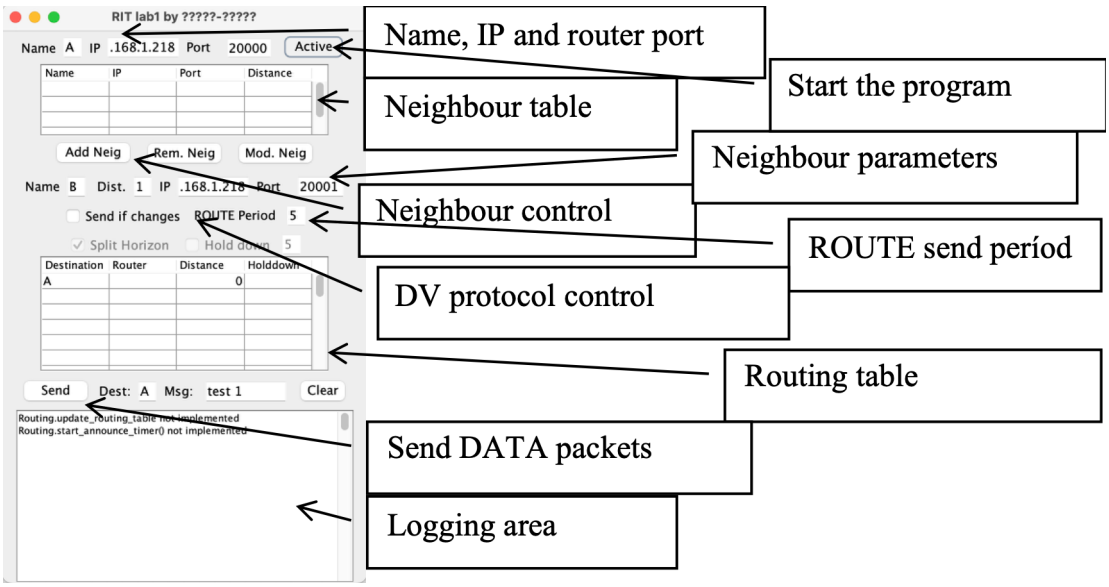
3.3. Provided code

To facilitate development and make it possible to complete the program in eight hours, we provide an incomplete router program along with the graphical interface shown below, which already performs some of the requested functions. Each group can make any modifications to the base program or even design a new graphical interface. However, it is recommended that you invest time in correctly implementing the routing algorithm.

---

[1] The instruction to create an account are in: https://docs.github.com/en/account-and-profile/how-tos/account-management/creating-an-account-on-github

[2] You can also clone via the command line if you install the git app (on Linux, MacOs, or Windows) and use the command "*git clone https://github.com/lflbernardofct/router_rit.git*".



The program provided is composed of ten classes:

- *Log.java* (complete) – Interface for logging messages on-screen;
- *Entry.java* (complete) – Element of neighbour array in ROUTE packet;
- *Neighbour.java* (complete) – Neighbour descriptor, used in neighbourhood control, storing distance vectors and packet forwarding;
- *NeighbourList.java* (complete) – Neighbour list descriptor, used in neighbourhood control and packet forwarding;
- *RouteEntry.java* (complete) – Store routing table elements and implements part of the hold down algorithms;
- *RoutingTable.java* (complete) – Stores a routing table. Offers functions to manipulate the table contents;
- *RouterInfo.java* (complete) – Descriptor of a router belonging to an area, with data received in the ROUTE packet;
- *LocalIPaddresses* (complete) – Management of the machine's IP addresses;
- *UnicastDeamon.java* (complete) – Unicast data reception thread;
- *Routing.java* (**to be completed**) – Class responsible for sending and receiving ROUTE packets, for calculation of the routing table and for handling DATA packets;
- *Router.java* (complete) – Main class with graphical interface (inherited from *JFrame*), which synchronises the various objects used; also manages the thread for receiving unicast data.

The provided program includes all complete files except *Routing.java*, which students must complete. Activation of sockets, data reception threads, and neighbourhood management is already done in the provided program. It is only necessary to implement the aspects related to the routing algorithm (control of the sending and processing of ROUTE packets, calculation of the routing table).

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

O programa fornecido inclui todos os ficheiros completos exceto o ficheiro *Routing.java* que deve ser completado pelos estudantes. A ativação dos sockets e da *thread* de receção de dados já é feita no programa fornecido. Falta apenas realizar os aspetos relacionados com o algoritmo de encaminhamento (controlo do envio e processamento de pacotes ROUTE, cálculo da tabela de

encaminhamento). **One of the main difficulties of the lab work is reading and understanding all the code provided so you can use it conveniently from the first lab class.**

The *Routing* class defines an initial set of fields with data obtained from the graphical interface, initialised in the class constructor:

```
private char local_name;      // Local name
private NeighbourList neig;   // Neighbour list
private int period;           // Routing update period (ms)
private boolean splitHorizon; // If it uses Split Horizon
private boolean holdown;      // If it uses hold down
private int holdown_time;     // Duration of hold down time (s)
private Router win;           // Main window
private DatagramSocket ds;    // Unicast datagram socket
private JTable tableObj;      // window with routing table
```

The *RoutingTable* class holds routing tables. Internally, it defines the field *rtab* where the routing table is stored:

```
private final ConcurrentHashMap<Character, RouteEntry> rtab;
```

The *RoutingTable* class defines the following methods to handle routing tables:

```
// Default constructor
public RoutingTable();

// Constructor that clones src contents
public RoutingTable(RoutingTable src);

// Test if table is initialized (it may be empty)
public boolean is_valid();

// Clear the table and stop holddown timers
public void clear();

// Add or replace a route entry to the table
public void add_route(RouteEntry re);

// Merges rt table to the local routing table
public void merge_table(RoutingTable rt);

// Delete a route entry from the table; returns true if done
public boolean delete_routeEntry(RouteEntry re);

// Returns the RouteEntry object to dest, or null if non-existing
public RouteEntry get_RouteEntry(char dest);

// Returns the next hop to reach dest, or ' ' if not existing
public char nextHop(char dest) {

// Returns an Entry[] with the table contents
public Entry[] get_Entry_vector();

// Returns a set with all the routes
public Collection<RouteEntry> get_routeset();

// Returns an iterator to go through the route entries
public Iterator<RouteEntry> iterator() {

// Compares rt with the routing table and returns true if equal
```

```
  public boolean equal_RoutingTable(RoutingTable rt);

  // Log the table contents using a log (Router)object
  public void Log_routing_table(Log log);
```

The *Routing* class defines the node's routing table, which is stored in the tab object.

```
  public RoutingTable tab;                    // Routing table object
```

During the work, the following *Routing* class methods must be defined or completed:

```
/** Sends ROUTE packets to all neighbors */
public boolean send_local_ROUTE() {
 … use the methods send_local_ROUTE_to_neighbour and
       prepare_vec_for_neighbour to send the packets … }

/** Creates the ROUTE Periodic Packet Send Clock */
private void start_announce_timer(int duration){ … }

/** It decodes ROUTE packets and processes them. The code is provided
    that decodes the packet. Packet processing is missing. */
public boolean process_ROUTE(char sender, DatagramPacket dp,
            String ip, DataInputStream dis) { … }

/** Calculates the forwarding table */
private synchronized void update_routing_table() { … }

/** Prepare the vector to send to neighbor n */
private Entry[] prepare_vec_for_neighbour(Neighbour n) {
 … prepara o vetor vec a enviar para um vizinho, tendo em conta os
   algoritmos de slip horizon e holddown … }

/** Handles hold down time end in route re */
public boolean handle_holddown_timeout(RouteEntry re) { … }
```

During the work, you can and should use the methods that are already available in the various classes of the work: to compare routing tables, compare vectors, etc. Thus, **before starting**, students should carefully read all the code provided so that they can take full advantage of it. Students can also modify the complete classes provided if they deem it necessary.

3.2 Goals

The recommended sequence for the development of the program is as follows:

1. Program an initial version of the *send_local_ROUTE* function that sends the ROUTE packet, in the *Routing class*;
2. Program in the *Routing* class the *start_announce_timer* function that implements the timer for sending ROUTE packets and starts the time count;
3. Read and understand the *Neighbor* class, and program the *process_ROUTE* function of the *Routing* class to store the distance vectors received in the neighbors' ROUTE packets in the neighbor list;
4. Programming the *Routing.update_routing_table* function to calculate the routing table by applying the simple distance vector algorithm;

git: create checkpoint "BasicDV"

5. Programming the algorithm variant with **Horizon Separation**;

git: create checkpoint "SplitHorizon"

6. Program the algorithm variant with ***Hold down***. For each destination, it should be checked if the previously existing route is no longer valid, and the route should be kept locked during the *hold down time*, until it searches for the destination again.

git: create checkpoint "Holddown"

ALL students should attempt to complete **at least goal 5**. In the first week of work, you should be working on goal 3. By the end of the second week, you should have finished goal 4. By the end of the third week, you should be working on goal 6. At the end of the fourth and final week, you should try to do as many goals as possible. Keep in mind that it is better to do less and do it well (working without errors and knowing the code delivered) than to do everything and have nothing work, or worse, not know the code delivered.

STUDENT POSTURE

Each group should consider the following:

- Don't waste time on data input and output aesthetics
- Program according to the general principles of good coding (use of indentation, comments, use of variables with names that conform to their functions ...)
- Ensure that the work to be done is equally distributed among the two group members
- The use of artificial intelligence tools in developing the code is allowed. The final oral discussion will assess the students' level of knowledge about the work delivered. It is better to deliver a more incomplete work, which you know well, than a very complete one that you do not know, which can lead to not being approved in the curricular unit.